SE101 – SOFTWARE ENGINEERING MODULE 4

1. Title of the Module: OBJECT ORIENTED SOFTWARE ENGINEERING: Behavioral UML Diagrams

2. Introduction

Software engineering is a discipline whose aim is the production of fault-free software that is delivered on time, within budget, and satisfies the user‟s needs. Software engineering is about teams. The problems to solve are so complex or large, that a single developer cannot solve them anymore. Software engineering is also about communication. Teams do not consist only of developers, but also of testers, architects, system engineers, customer, project managers, etc

3. Learning Outcome

After completing this module, the students must be able to:

- **Prepare following diagrams for selected project:**

1. Use case diagram
2. Activity diagram
3. Sequence diagram
4. State diagram
5. Communication diagram
6. Interaction overview diagram
7. Timing diagram

4. Learning Content

1. Use case diagram

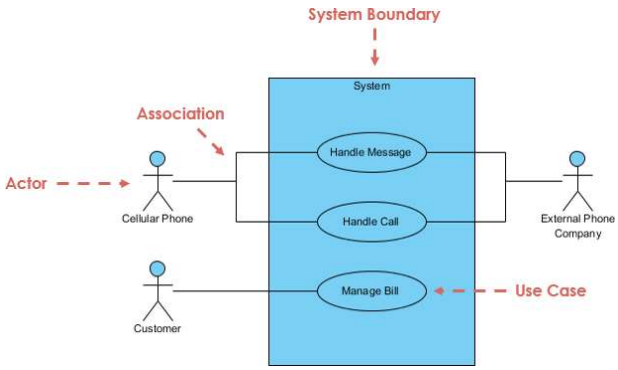
Purpose of Use Case Diagram

Use case diagrams are typically developed in the early stage of development and people often apply use case modeling for the following purposes:

- Specify the context of a system
- Capture the requirements of a system
- Validate a systems architecture
- Drive implementation and generate test cases
- Developed by analysts together with domain experts

A standard form of use case diagram is defined in the Unified Modeling Language as shown in the Use Case Diagram example below:

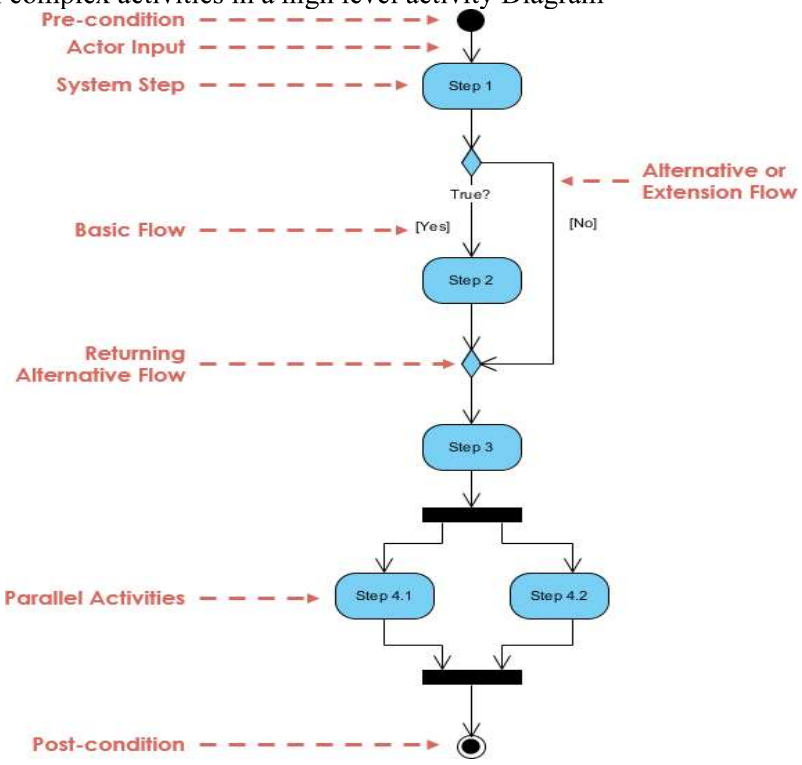
- Actor
- Use Case
- Communication Link
- Boundary of system



USE Case Relationship:

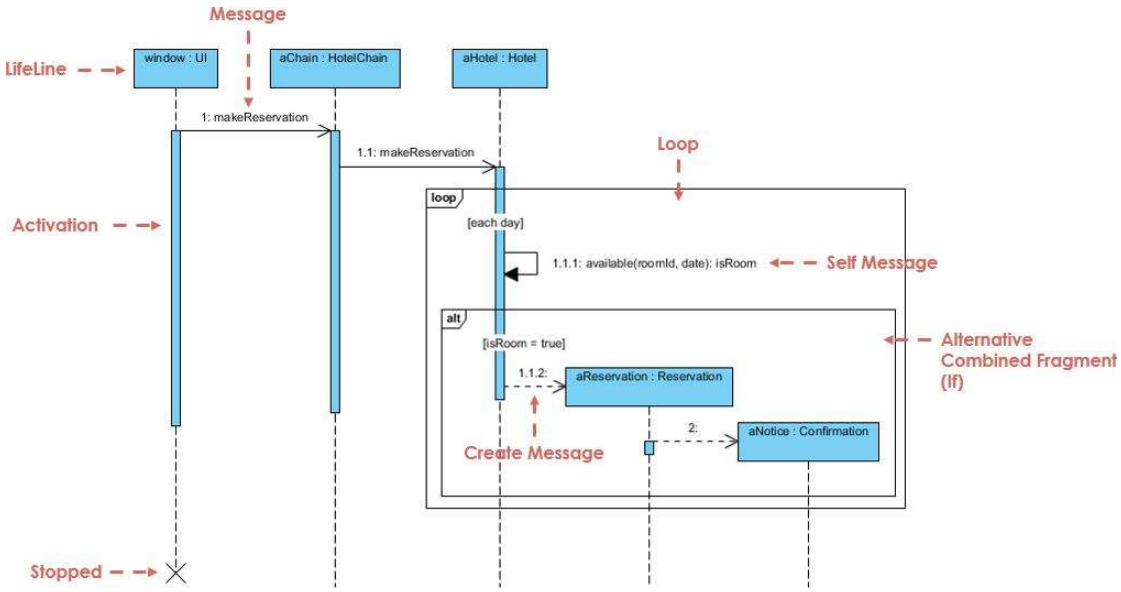
- **Extends**
- **Include**
- **Generalization**

2. **Activity diagram**
   - Identify candidate use cases, through the examination of business workflows
   - Identify pre- and post-conditions (the context) for use cases
   - Model workflows between/within use cases
   - Model complex workflows in operations on objects
   - Model in detail complex activities in a high level activity Diagram



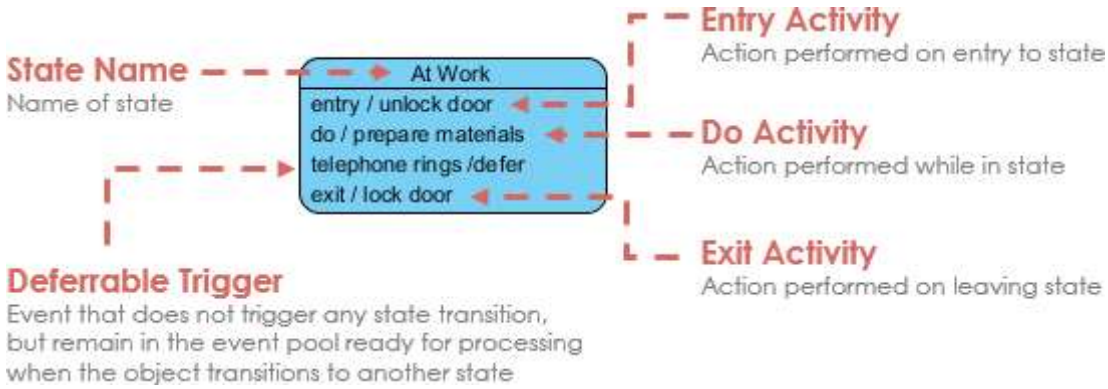
3. **Sequence diagram**
   - Model high-level interaction between active objects in a system
   - Model the interaction between object instances within a collaboration that realizes a use case
   - Model the interaction between objects within a collaboration that realizes an operation
   - Either model generic interactions (showing all possible paths through the interaction) or specific instances of a interaction (showing just one path through the interaction)

     Example :Hotel Reservation



4. **State diagram**

State machine diagram typically are used to describe state-dependent behavior for an object. **An object responds differently to the same event depending on what state it is in**. State machine diagrams are usually applied to objects but can be applied to any element that has behavior to other entities such as: actors, use cases, methods, subsystems systems and etc. and they are typically used in conjunction with interaction diagrams (usually sequence diagrams)



States:

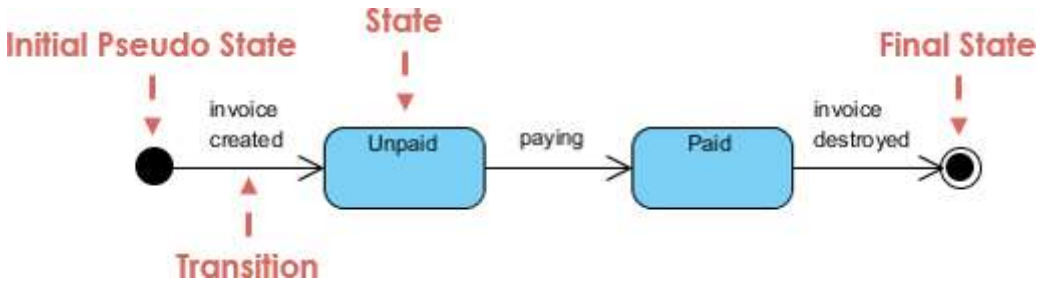
1. Initial State-Entry State
2. Final State -Exit State

Events :

1. Signal event - corresponding to the arrival of an asynchronous message or signal
2. Call event - corresponding to the arrival of a procedural call to an operation
3. Time event - a time event occurs after a specified time has elapsed
4. Change event - a change event occurs whenever a specified condition is met

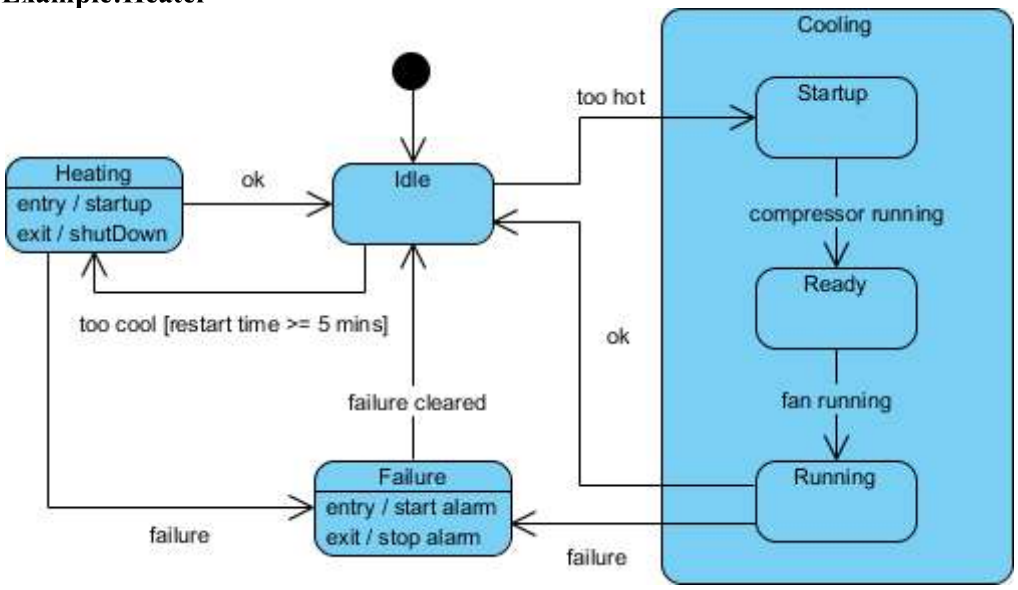
Transition

1. An element is in a source state
2. An event occurs
3. An action is performed
4. The element enters a target state



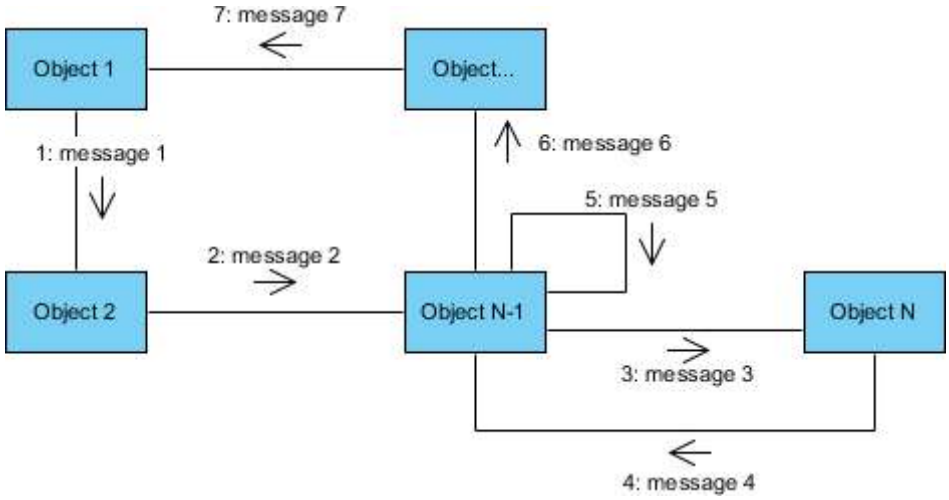
Substates:
A simple state is one which has no substructure. A state which has substates (nested states) is called a composite state. Substates may be nested to any level. A nested state machine may have at most one initial state and one final state

Example:Heater

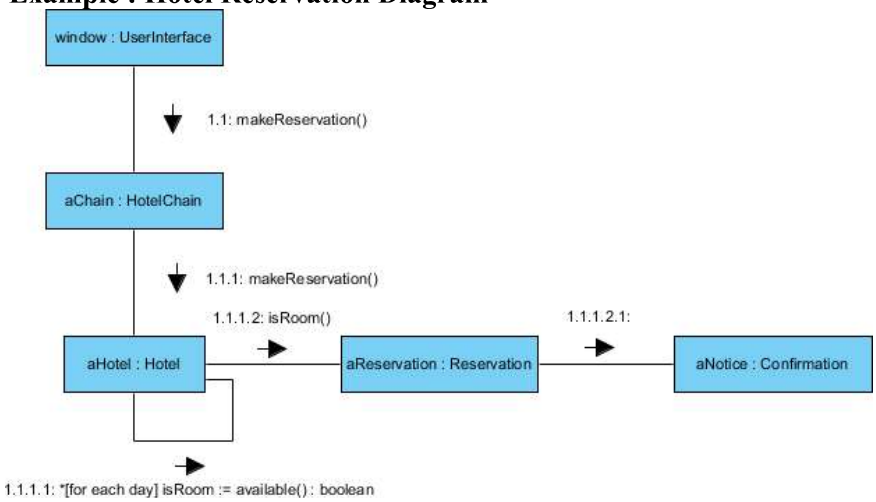


5. Communication diagram

- Model message passing between objects or roles that deliver the functionalities of use cases and operations
- Model mechanisms within the architectural design of the system
- Capture interactions that show the passed messages between objects and roles within the collaboration scenario
- Model alternative scenarios within use cases or operations that involve the collaboration of different objects and interactions
- Support the identification of objects (hence classes), and their attributes (parameters of message) and operations (messages) that participate in use cases
- Each message in a communication diagram has a sequence number.
- The top-level message is numbered 1.
- Messages sent during the same call have the same decimal prefix, but suffixes of 1, 2, etc. according to when they occur.

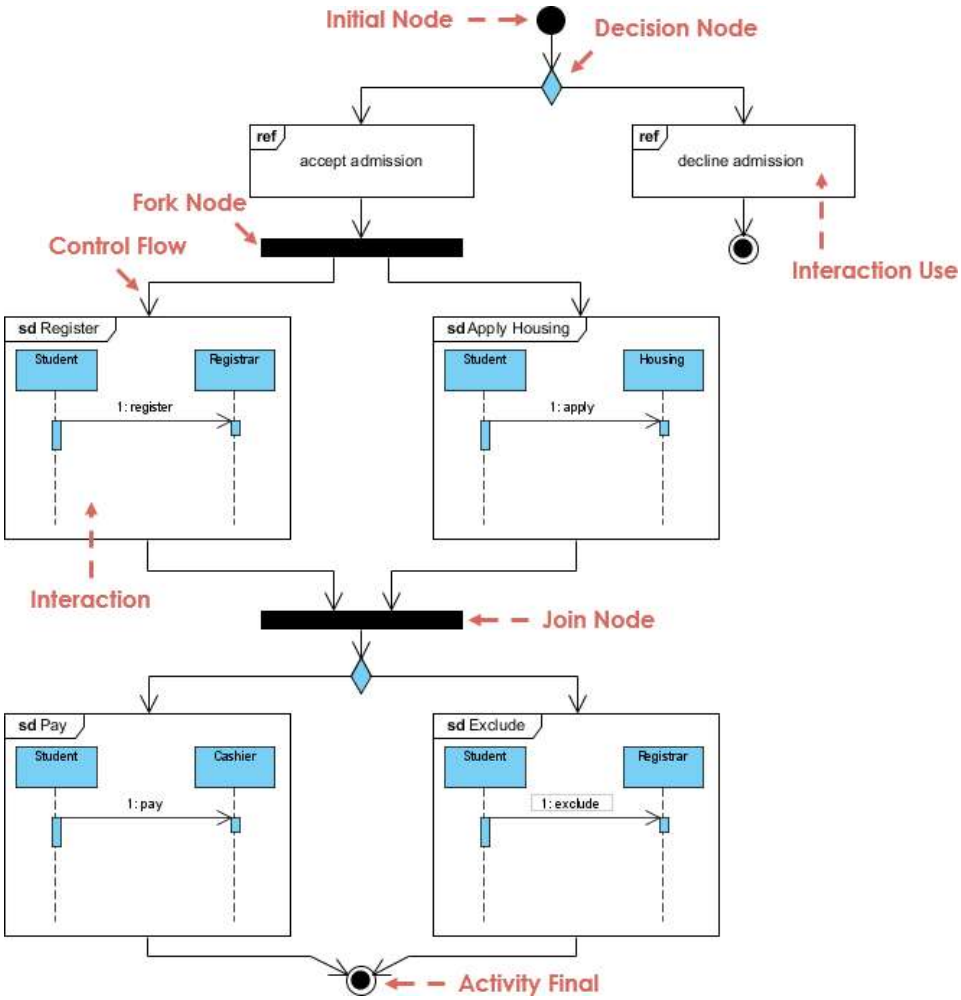


Example : Hotel Reservation Diagram



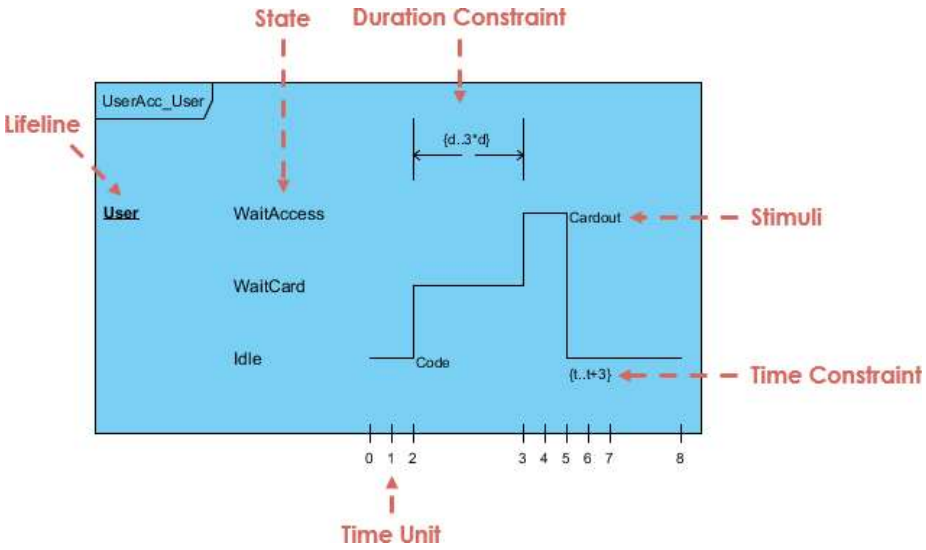
6. **Interaction overview diagram**

a student who has been accepted into a university. First the student must be accept or decline admission. After accepting, the student must both register for classes and apply for housing. After both of those are complete, the student must pay the registrar. If payment is not received in time the student is excluded by the registrar.

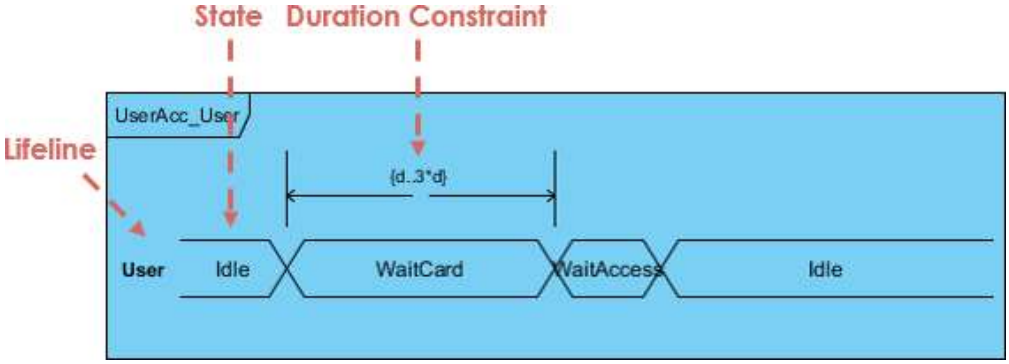


7. Timing diagram

Changes from one **state** to another are represented by **a change in the level of the lifeline**. For the period of time when the object is a given state, the timeline runs parallel to that state. A change in state appears as a vertical change from one level to another. The cause of the change, as is the case in a state or sequence diagram, is the receipt of a message, an event that causes a change, a condition within the system, or even just the passage of time.

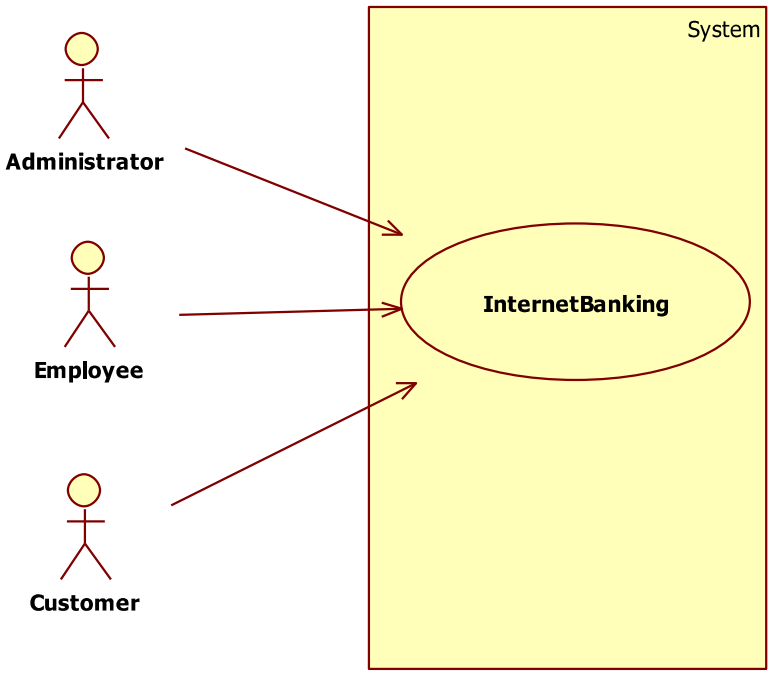


Value Lifeline



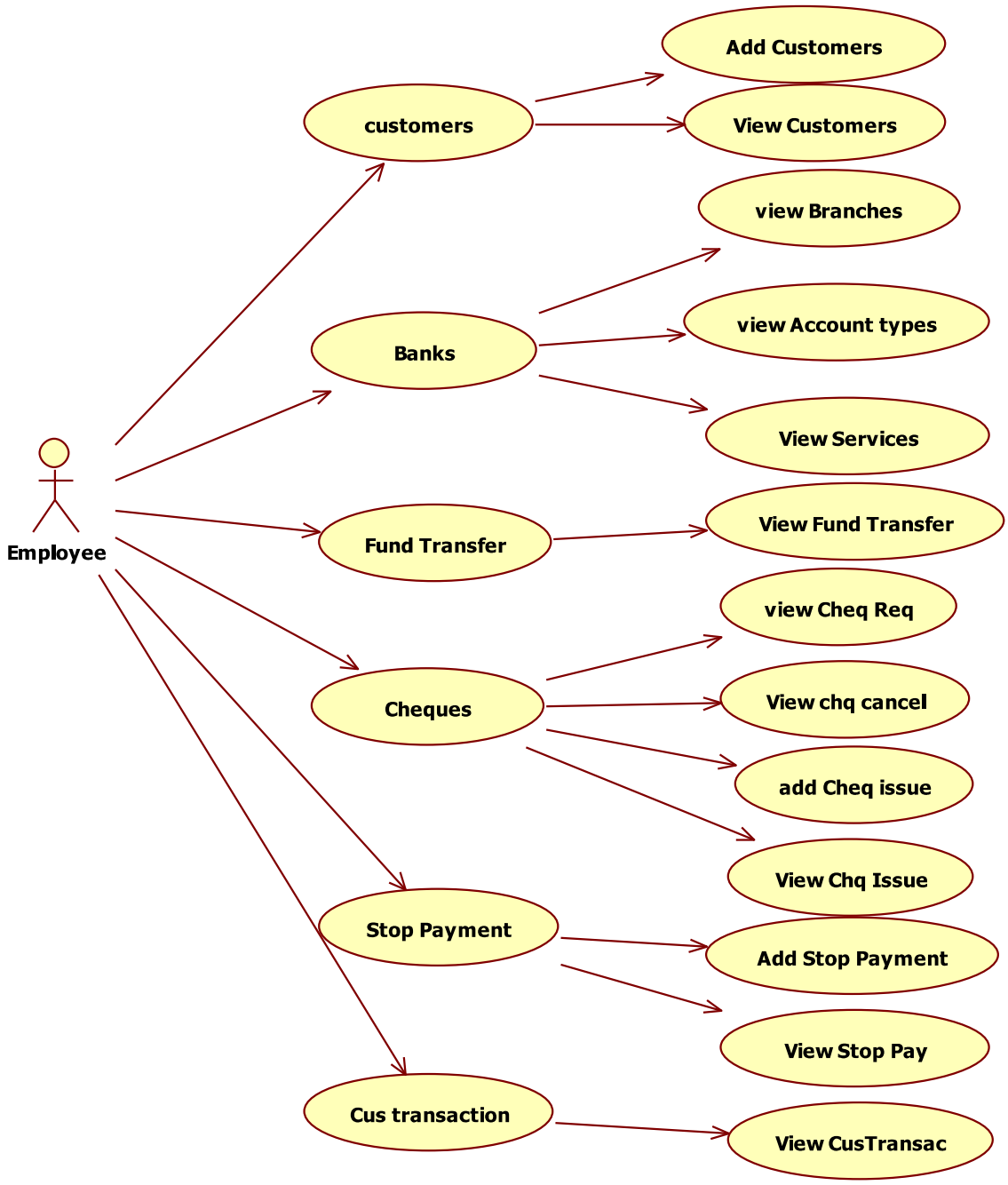
1) Use-Case Diagram:

System use case Diagram:



Administrator Use case Diagram:

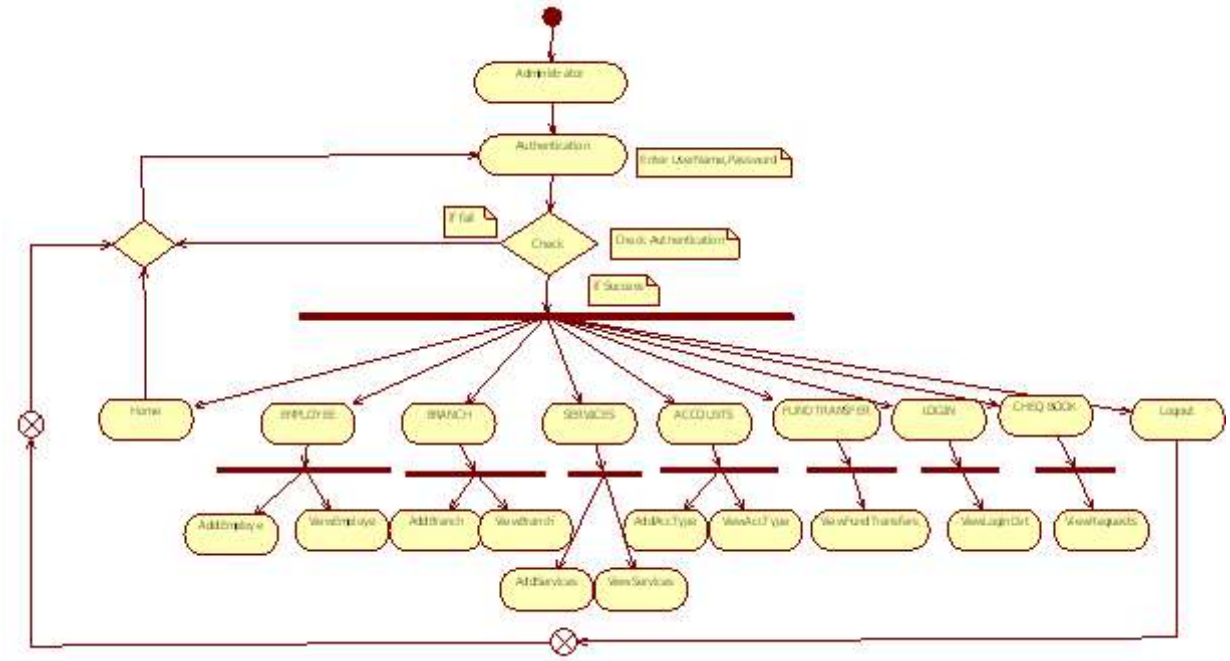
Employee Use case Diagram:



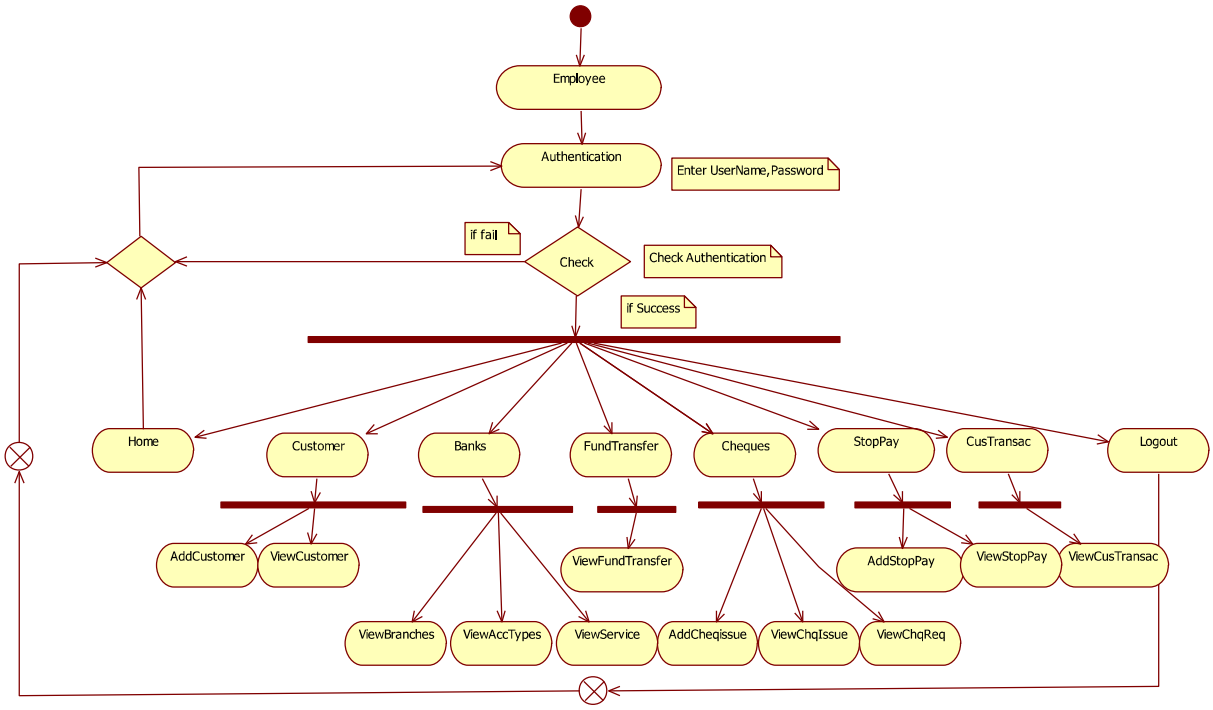
Customer Use Case Diagram:

2) Activity Diagram:

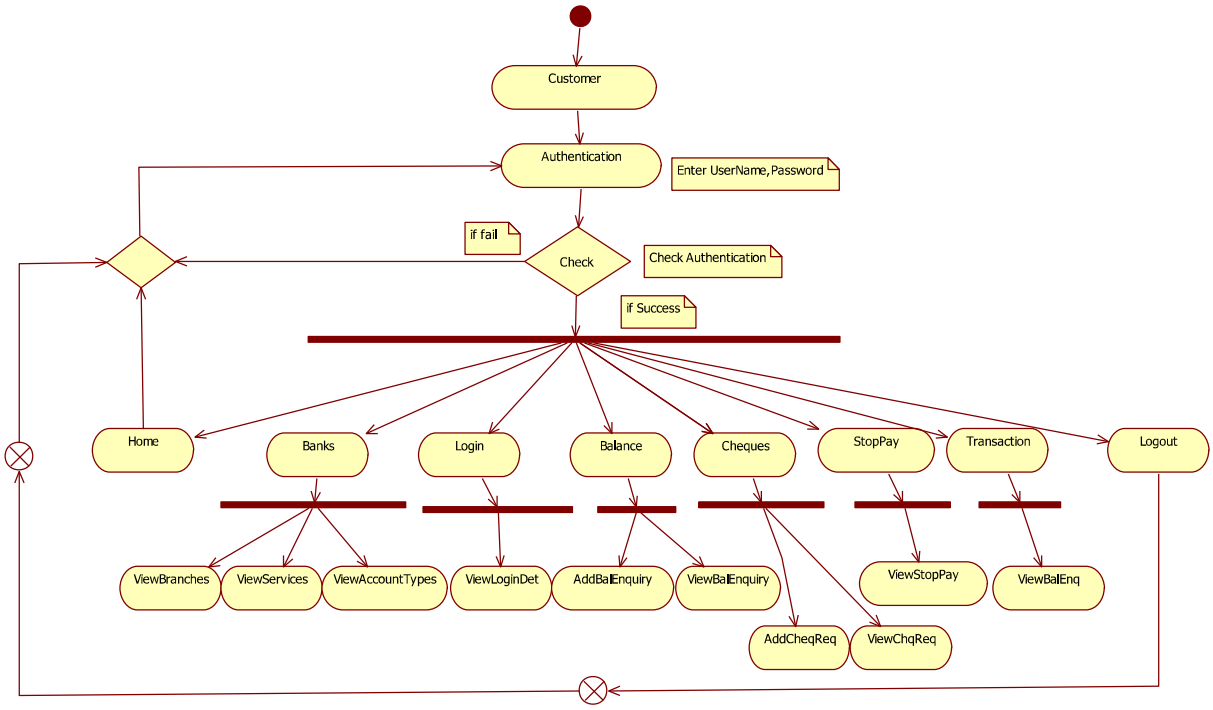
Administrator Activity Diagram:



Employee Activity Diagram:

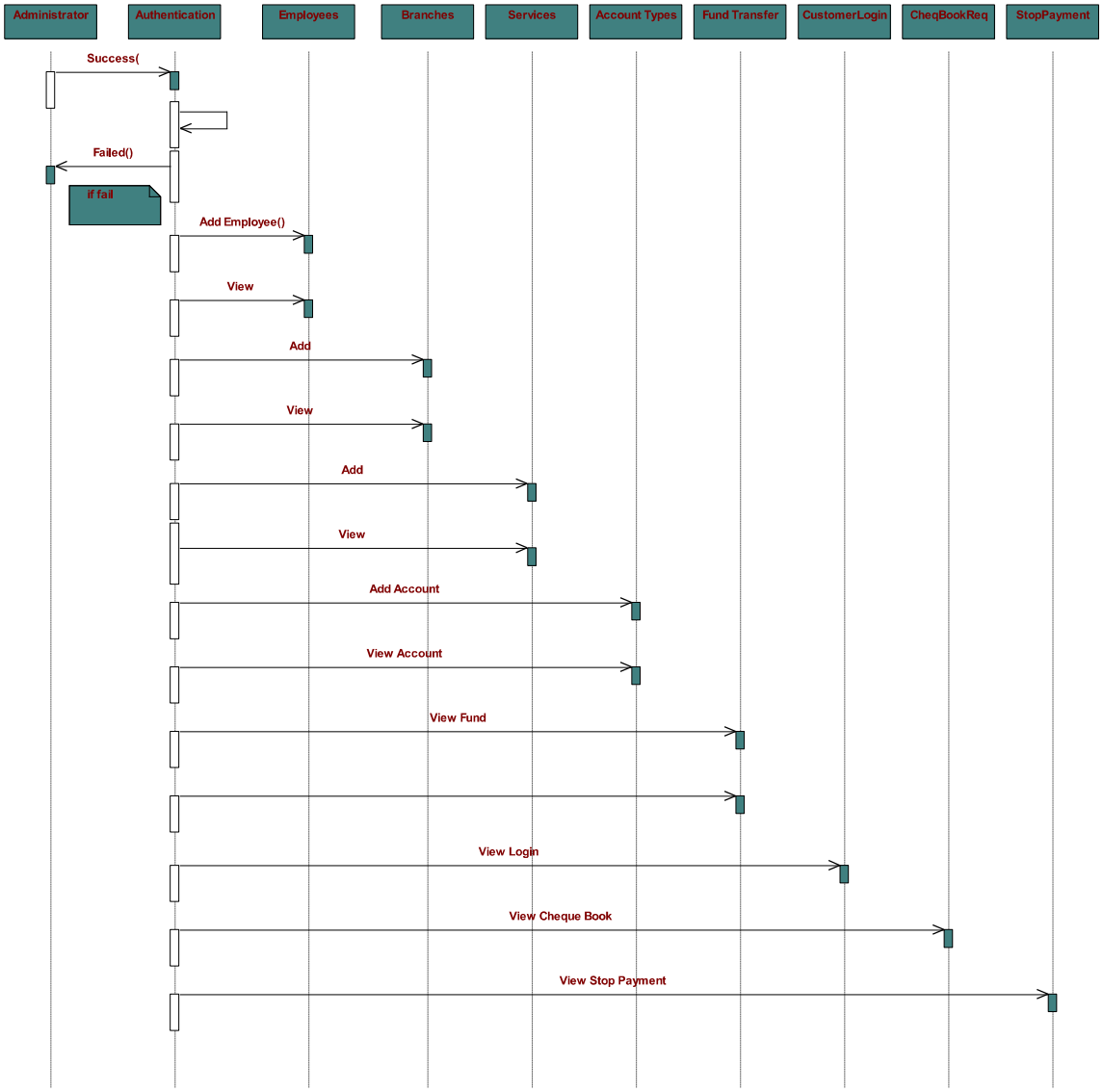


Customer Activity Diagram:

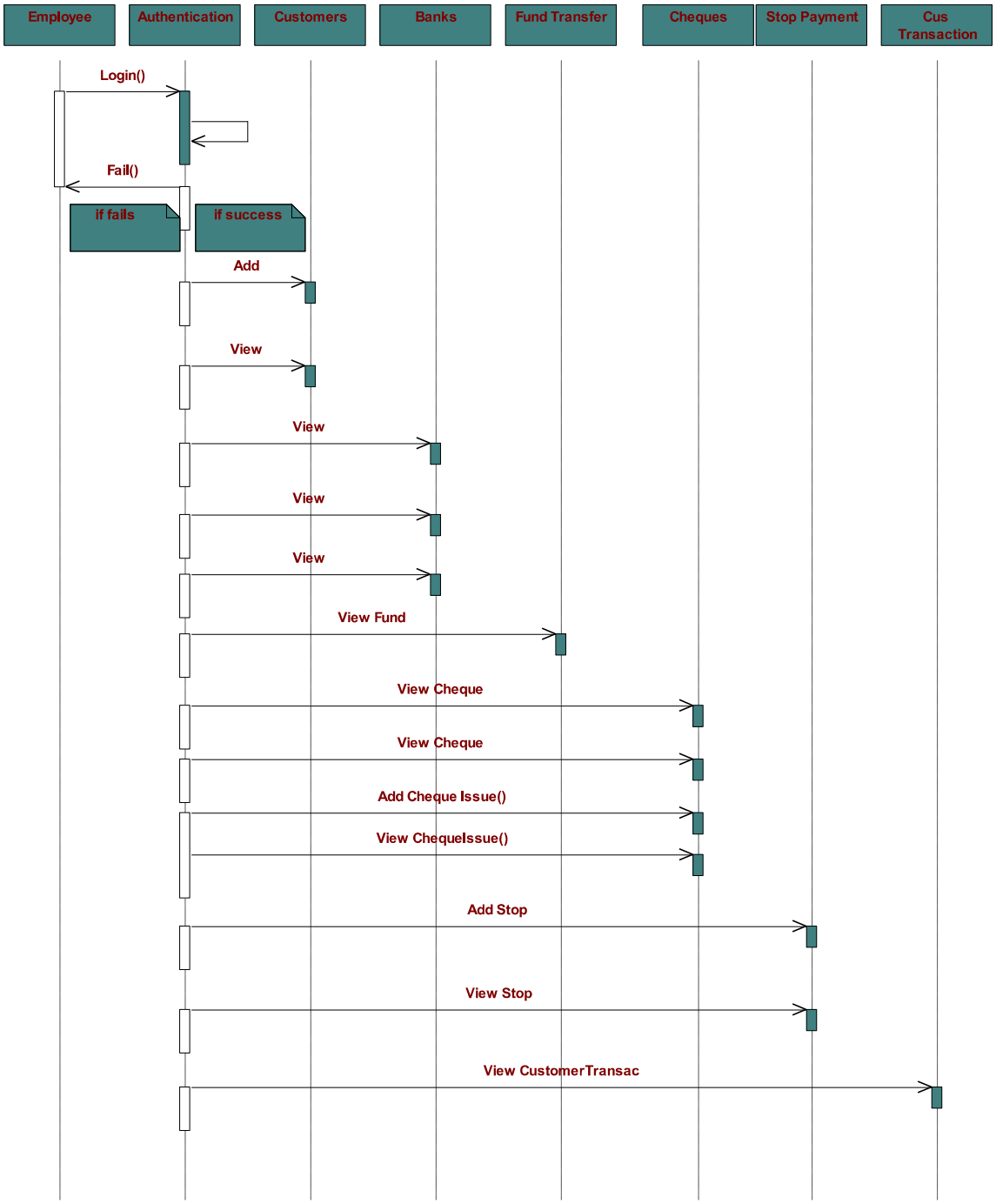


3) Sequence Diagram:

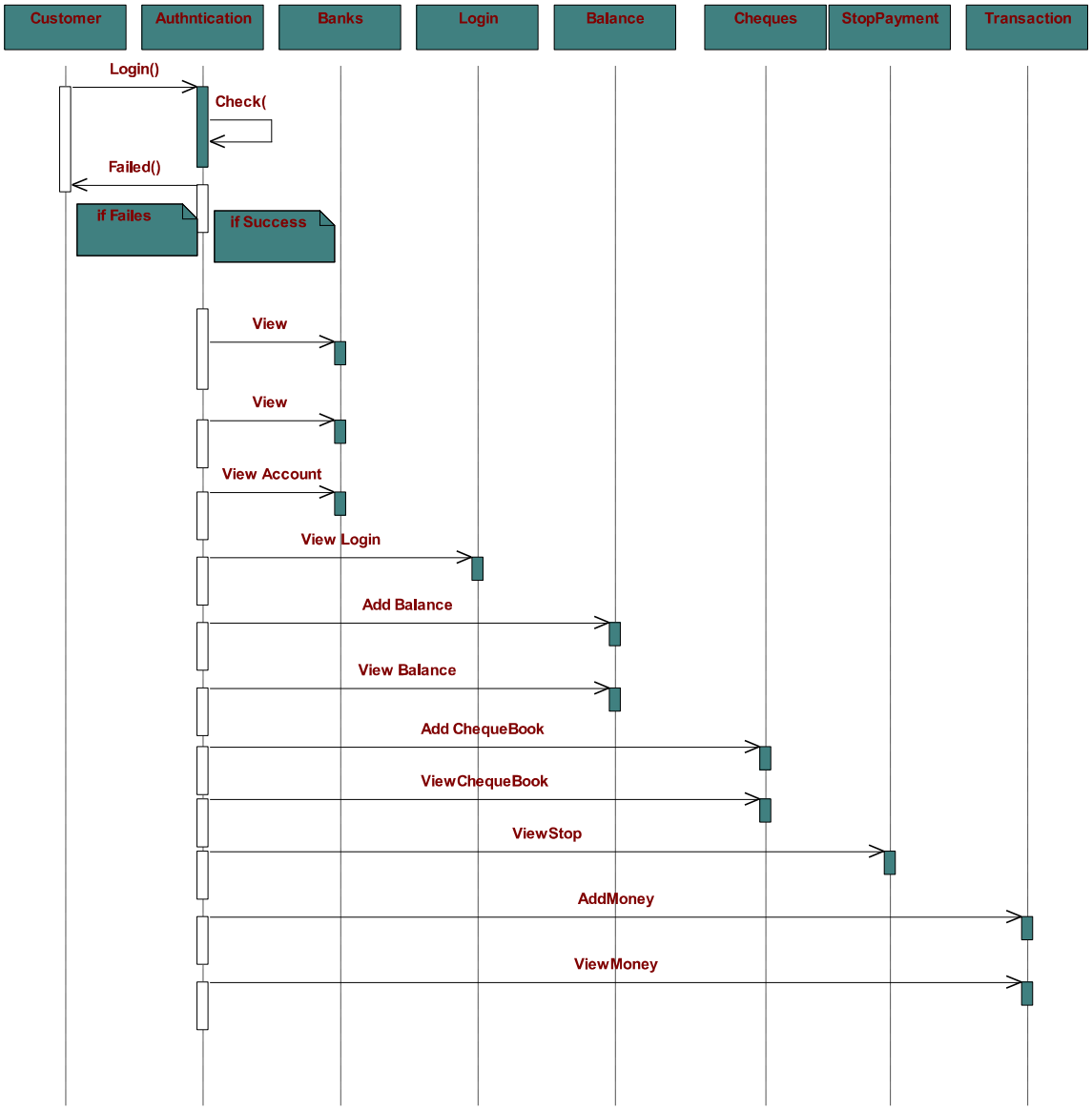
Administrator Sequence Diagram:



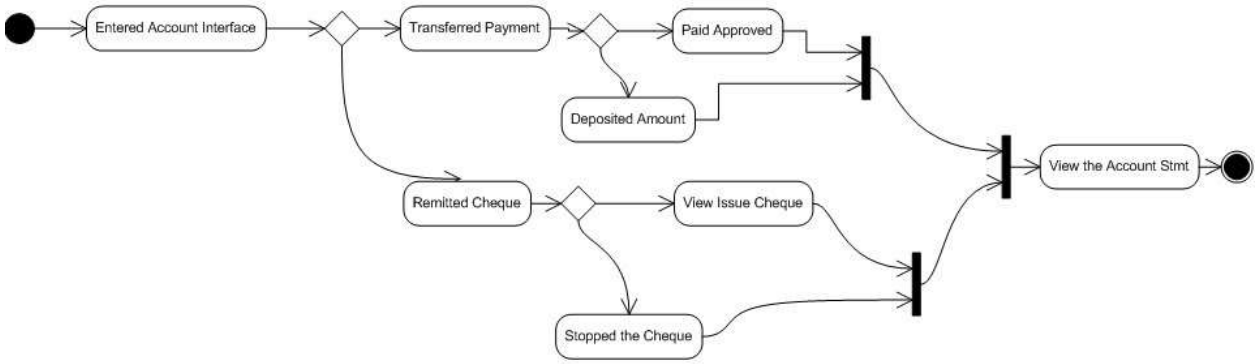
Employee Sequence Diagram:



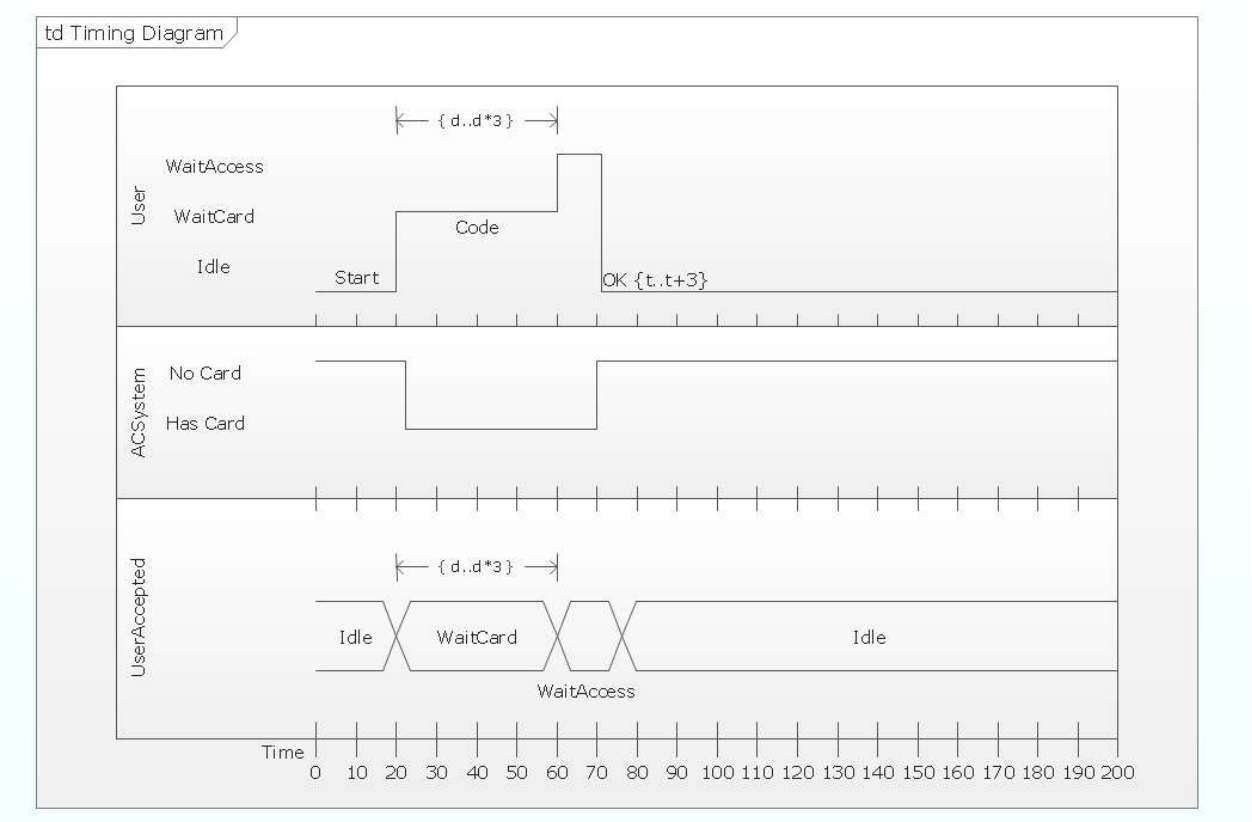
Customer Sequence Diagram:



4) State Diagram:



5) Timing Diagram :



5. Teaching and Learning Activities

Prepare the following documents for each experiment and develop the software using software engineering methodology.

1. Problem Analysis and Project Planning -Thorough study of the problem –Identify Project scope, Objectives and Infrastructure.

2. Software Requirement Analysis - Describe the individual Phases/modules of the project and Identify deliverables.

3. Data Modelling - Use work products – data dictionary, use case diagrams and activity diagrams, build and test class diagrams, sequence diagrams and add interface to class 4. diagrams.

5. Software Development and Debugging – implement the design by coding

6. Software Testing - Prepare test plan, perform validation testing, coverage analysis, memory leaks, develop test case hierarchy, Site check and site monitor**.**

6. Recommended learning materials and resources for supplementary reading.

All related Software Engineering books are recommended for reading. Also take time to read Data Privacy Act of the Philippines.

Software Required: MS Word, Opensource tools: UMLet

7. Flexible Teaching Learning Modality (FTLM) adopted Online (synchronous)

Facebook messenger, Zoom, Google Classroom, etc.. Remote (asynchronous) module

8. Assessment Task Recitation Quizzes (Separate activity sheet is provided)

9. References

Peleeger, S.L, SOFTWARE ENGINEERING: Theory and Practices. Second Edition. Prentice_hall Inc. 2001

Prepared by:

CAROLINE B. PAULE, MECpE, LpT

FACULTY